

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего  
образования «Самарский национальный исследовательский университет имени академика  
С.П. Королева»  
(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

## ОТЧЕТ ПО ПРАКТИКЕ

Вид практики производственная  
(учебная, производственная)

Тип практики научно-исследовательская работа  
(в соответствии с ОПОП ВО)

Сроки прохождения практики: с 01.10.2024 по 30.12.2024  
(в соответствии с календарным учебным графиком)

по направлению подготовки 09.03.01 Информатика и вычислительная техника  
(уровень бакалавриата)  
направленность (профиль) «Информационные системы»

Обучающийся группы № 6304-090301D \_\_\_\_\_ М.П. Федулов

Руководитель практики,  
доцент кафедры ИСТ, к.т.н. \_\_\_\_\_ Я.В. Соловьева

Дата сдачи 30.12.2024

Дата защиты 30.12.2024

Оценка \_\_\_\_\_

Самара 2024

## **СОДЕРЖАНИЕ**

1. Задание(я) для выполнения определенных видов работ, связанных с будущей профессиональной деятельностью (сбор и анализ данных и материалов, проведение исследований).
2. Описательная часть.
3. Список использованных источников.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение  
высшего образования «Самарский национальный исследовательский университет  
имени академика С.П. Королева»  
(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

**Задание(я) для выполнения определенных видов работ, связанных с  
будущей профессиональной деятельностью (сбор и анализ данных и  
материалов, проведение исследований)**

Обучающемуся Федулову Михаилу Павловичу  
группы 6304-090301D

Направление на практику оформлено приказом по университету  
от 27.09.2024 г. № 450-ПР

на кафедру информационных систем и технологий  
(наименование профильной организации или структурного подразделения университета)

Тема НИР: «Исследование подходов разработки, взаимодействия и  
сборки микрофронтенд приложений»

| Планируемые результаты освоения образовательной программы (компетенции)                                                                                                                                                                                                                                              | Планируемые результаты практики                                                                                                                                                                                                                                                                 | Содержание задания                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ПК-7 Способен осуществлять выполнение работ и управление работами по созданию (модификации) и сопровождению информационных систем, автоматизирующих задачи организационного управления и бизнес-процессы.<br>ПК-7.1 Осуществляет выявление, анализ, согласование и утверждение требований к информационным системам. | Знать: методики анализа, согласования и утверждения требований к информационным системам.<br>Уметь: применять методики анализа, согласования и утверждения требований к информационным системам.<br>Владеть: навыками анализа, согласования и утверждения требований к информационным системам. | Провести анализ требований к разрабатываемой информационной системе разработки, взаимодействия и сборки микрофронтенд приложений.<br><br>Согласовать и утвердить требования к разрабатываемой информационной системе разработки, взаимодействия и сборки микрофронтенд приложений.<br><br>Провести анализ имеющихся систем-аналогов. |
| ПК-7 Способен осуществлять выполнение работ и управление работами по созданию                                                                                                                                                                                                                                        | Знать: технологии проектирования архитектуры информационных систем.                                                                                                                                                                                                                             | Провести анализ имеющихся технологий проектирования архитектур информационных систем и сделать обоснование                                                                                                                                                                                                                           |

|                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (модификации) и сопровождению информационных систем, автоматизирующих задачи организационного управления и бизнес-процессы.<br>ПК-7.2 Разрабатывает архитектуру информационных систем.                                                                                        | Уметь: применять технологии проектирования для разработки информационно-логического проекта информационной системы.<br>Владеть: навыками разработки информационно-логических проектов информационных систем.               | выбора используемой технологии.<br><br>Разработать информационно-логический проект информационной системы разработки, взаимодействия и сборки микрофронтенд приложений.<br><br>Сделать описание информационно-логического проекта информационной системы разработки, взаимодействия и сборки микрофронтенд приложений. |
| ПК-7 Способен осуществлять выполнение работ и управление работами по созданию (модификации) и сопровождению информационных систем, автоматизирующих задачи организационного управления и бизнес-процессы.<br>ПК-7.4 Проектирует и разрабатывает дизайн информационных систем. | Знать: технологии проектирования дизайна информационных систем.<br>Уметь: применять технологии проектирования для разработки дизайна информационных систем.<br>Владеть: навыками разработки дизайна информационных систем. | Провести анализ имеющихся технологий проектирования дизайна информационных систем и сделать обоснование выбора используемой технологии.<br><br>Разработать дизайн информационной системы разработки, взаимодействия и сборки микрофронтенд приложений.                                                                 |

Дата выдачи задания 01.10.2024.

Срок представления на кафедру отчета о практике 30.12.2024.

Руководитель практики,

доцент кафедры ИСТ, к.т.н. \_\_\_\_\_ Я.В. Соловьева  
(подпись)

Задание принял к исполнению

обучающийся группы № 6304-090301D \_\_\_\_\_ М.П. Федулов  
(подпись)

## Описательная часть

### 1. Введение

Веб-сайт (далее сайт) — это набор логически связанных между собой веб-страниц, расположенных в Сети под единым именем — доменом. Веб-страницы — это документы, которые написаны на языке разметки HTML и доступны в сети Интернет. Веб-страницы содержат контент — например, статьи, фото или видео. [1]

Каждый современный сайт состоит из двух главных «слоёв»: frontend и backend.

Фронтенд (англ. Frontend) – это разработка пользовательских функций и интерфейса. К нему можно отнести всё, что пользователи видят на сайте или в веб-приложении, и с чем можно взаимодействовать (например, различные кнопки и списки). Внешний вид любого сайта – это и есть работа фронтенд-разработчика.

Бэкэнд (англ. Backend) – это логика работы сайта, скрытая от пользователя. В отличие от фронтенда, код работы сайта не отображается в браузере, а выполняется на сервере, где находится приложение или сайт. Пользователь взаимодействует с фронтендом и дальнейшие действия происходят в бекэнде. [2]

Эти две части взаимосвязаны, и одно не будет работать без другого: нажатие на кнопку не будет ничего делать или результат работы кода не будет отображаться вообще.

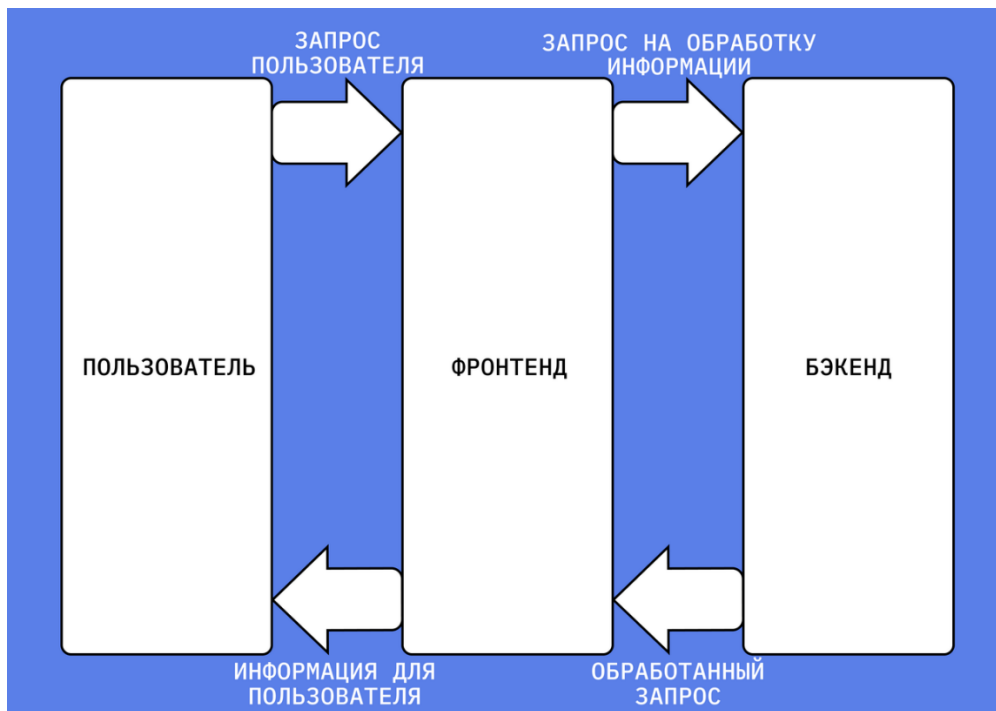


Рисунок 1 – Демонстрация взаимодействия пользователя с фронтом и бэкэндом.

Во время разработки и дальнейшей поддержке какого-либо проекта, можно столкнуться с рядом различных проблем, например, чрезмерный размер проекта. На начальном этапе эта проблема не особо заметна, её последствия начинают проявляться с добавлением всё нового и нового функционала. Следовательно, проект начинает сильно расширяться и превращаться в некий своего рода «монолит».

Монолитная архитектура представляет собой традиционный подход к разработке проекта, при котором все компоненты приложения объединены в единый, неразделимый блок-монолит. В такой системе пользовательский интерфейс, бизнес-логика и доступ к данным тесно связаны и функционируют как одно целое.

У данного подхода есть много преимуществ: простота разработки и тестирования, упрощенное развертывание, высокая производительность. К сожалению, такая архитектура не идеальна и имеет некоторые недостатки: сложность масштабирования, трудности в поддержке и обновлении, ограниченная гибкость.

Для решения проблемы с объемом проекта можно применять микрофронтендный подход.

Микрофронтенд – это разделение монолитного фронта на отдельные кодовые базы, хранящиеся в отдельных репозиториях. По большей части этот подход представляет собой организационное решение о том, как управлять сложностью разработки в большом проекте. Огромный «монолит» приложения можно разбить на несколько микрофронтендов, где каждая команда разработчиков будет отвечать за свою часть.

Стоит учесть, что приложение хоть и делится на несколько частей, важно не отходить от единого стиля и концепции всего «монолита».

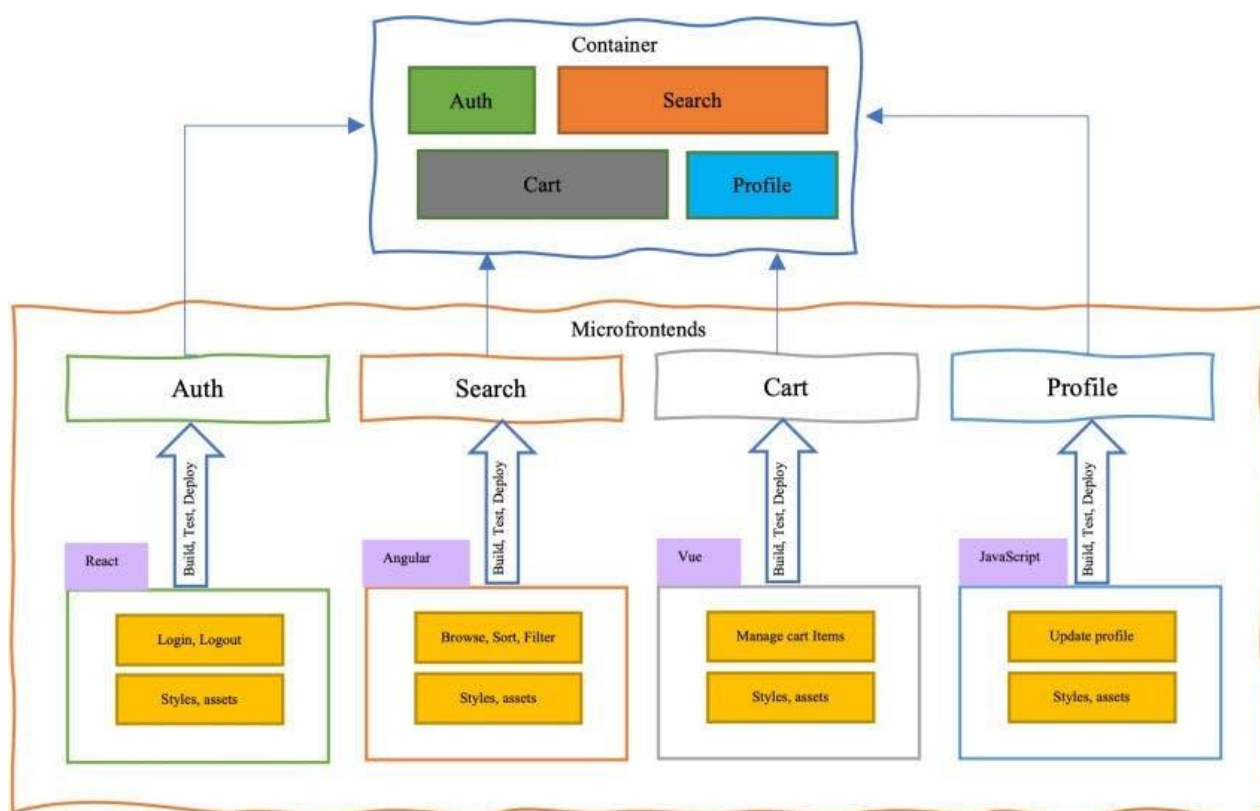


Рисунок 2 – Демонстрация разделения проекта на части-микрофронтенды.

Некоторые реализации микрофронтенда могут, как и ускорить разработку, так и замедлить. Поэтому важно выбрать конкретную реализацию в зависимости от нужд проекта.

## 2. Подходы разработки микрофронтенд приложений

В данный момент, существуют подходы разработки микрофронтенд приложений, такие как: Iframe, Podium, Single-SPA, Module Federation.

Рассмотрим каждый подход подробнее.

### 2.1. Iframe

Iframe – встраивание независимого приложения через специальный HTML-тег `<iframe>`. Это один из самых старых и простых способов организации микрофронта. В этой реализации получается своего рода «сайт в сайте». К преимуществам можно отнести отсутствие привязки к фреймворкам и полное изолирование логики и стиля. К недостаткам же можно отнести неспособность поисковых роботов отрисовки Iframe для последующего индексирования (проблемы с SEO).

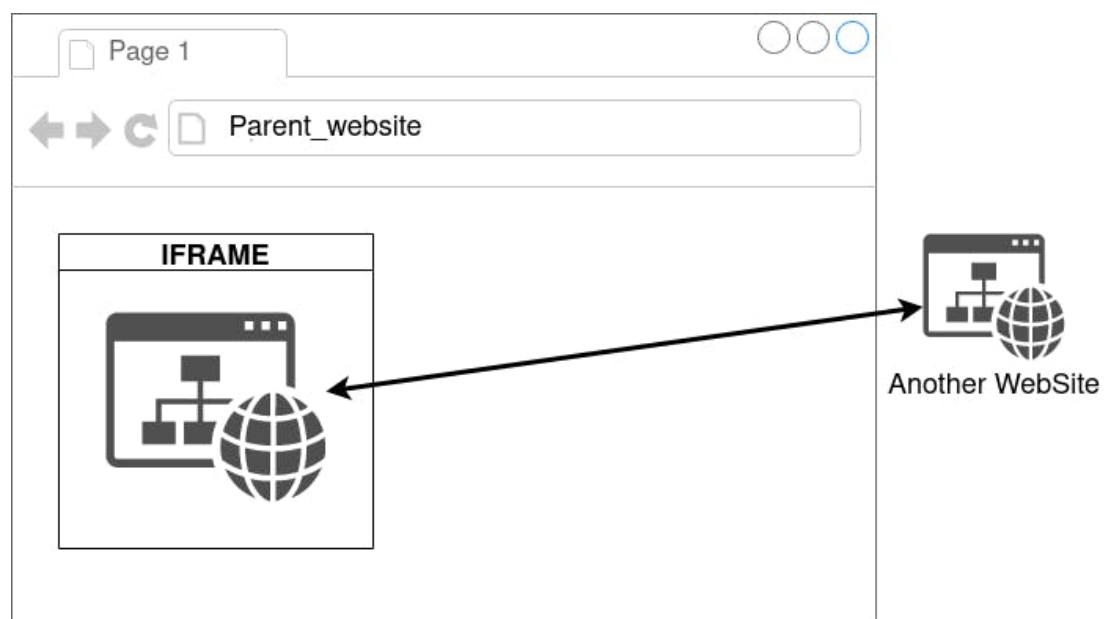


Рисунок 3 – Демонстрация работы Iframe.

Важно отметить, так как здесь происходит встраивание отдельных элементов одного сайта в другой, можно столкнуться с проблемой несостыковки стилей сайтов – если стиль встраиваемого сайта будет немного отличаться, эти отличия будут заметны и на основном сайте (которые, скорее всего, придется корректировать).

Преимущества:

- простота реализации
- логика и изоляция стилей
- независимое развертывание

Недостатки:

- SEO (Search Engine Optimization)
- возможное расхождение в стиле

## 2.2. Podium

Podium – это фреймворк для серверной композиции микрофронтендов, разработанный норвежской компанией Finn.no для их сайта объявлений [5].

Фреймворк (с англ. Framework – «каркас, структура») – заготовка, готовая модель в программировании для быстрой разработки. Эта заготовка задает структуру, определяет правила и предоставляет необходимый набор инструментов для создания проекта.

Теперь рассмотрим подробнее сам фреймворк.

Данный фреймворк позволяет объединять независимые части сайта (podlets) в единое целое на сервере. Podlets (далее подлеты) – самостоятельные HTTP-сервисы, разработанные и запущенные изолированно, которые могут быть написаны на любом языке программирования. Основными особенностями являются сами подлеты, возможность композиции на сервере и взаимодействие между подлетами на контрактной основе (определенные правила взаимодействия). [6]

Преимущества:

- автономная разработка
- возможность реализации подлетов на различных языках и фреймворках
- масштабируемость и независимое развертывание

Недостатки:

- необходимость в наличии сервера на Node.js

### 2.3. Single-SPA

Single-SPA – это JavaScript-фреймворк, предназначенный для объединения нескольких микрофронтендов в одно веб-приложение (Single Page Application – SPA). Он позволяет использовать различные фреймворки и библиотеки (например, Angular, React, Vue, и др.) в рамках одного проекта, обеспечивая совместную работу и независимую разработку.

Рассмотрим каждый фреймворк подробнее.

#### 2.3.1. Angular

Angular – это открытый фреймворк для создания SPA с использованием HTML и TypeScript. Был выпущен компанией Google в 2016 году. [7]

Основными особенностями являются:

- компонентная архитектура: приложение строится из компонентов со своей логикой, шаблонами и стилями.
- двусторонняя привязка данных: обеспечение синхронизации между моделью и представлением.
- внедрение зависимостей: встроенная система позволяет легко управлять зависимостями и сервисами.
- роутинг: возможность создания многостраничного интерфейса в рамках одного приложения с поддержкой навигации.

- инструменты разработки: Angular CLI представляет собой набор команд для создания, сборки, тестирования и развертывания приложения, сильно упрощая процесс разработки.

К преимуществам фреймворка можно отнести масштабируемость (можно разрабатывать как небольшие, так и крупные корпоративные приложения) и активную поддержку. [8]

Сам Angular был основан на более старой версии фреймворка – AngularJS. AngularJS был написан на языке программирования JavaScript, в отличие от новой версии на TypeScript. [7]

### 2.3.2. React

React – это JavaScript-библиотека с открытым исходным кодом, предназначенная для разработки пользовательских интерфейсов, в частности одностраничных приложений (SPA). Была разработана компанией Facebook и впервые использована в 2011 году. [9]

Основные особенности React:

- компонентный подход: React позволяет разбивать интерфейс на независимые компоненты.

- виртуальный DOM: использование виртуального DOM для оптимизации обновлений пользовательского интерфейса. DOM (Document Object Model) – это объектная модель документа, которую браузер создает в памяти компьютера на основании HTML-кода, полученного с сервера.

- однонаправленная передача данных: данные передаются сверху вниз (от родительских компонентов к дочерним), что обеспечивает предсказуемость поведения приложения и упрощает отладку.

- JSX (JavaScript XML): расширение синтаксиса JavaScript, которое позволяет писать HTML-подобный код внутри JavaScript. Это делает код более читаемым и упрощает создание новых компонентов.

- методы жизненного цикла: данные методы позволяют разработчику запускать код на различных стадиях жизненного цикла компонента.

- React Hooks: специальные функции, позволяющие использовать состояние и другие возможности React в компонентах без необходимости писать классы. Это позволяет упрощать код и повторно использовать логику с состоянием между различными компонентами без необходимости менять их структуру. [9]

К основным преимуществам React можно отнести высокую производительность, гибкость и модульность, активную поддержку.

Эта библиотека также может применяться в мобильной разработке с использованием фреймворка React Native.

### 2.3.3. Vue

Vue – это JavaScript-фреймворк с открытым исходным кодом, предназначенный для разработки пользовательских интерфейсов и одностраничных приложений (SPA). Был разработан Эваном Ю и впервые выпущен в феврале 2014 года. [11]

Основные особенности Vue:

- реактивность: данный фреймворк автоматически отслеживает изменения в данных и обновляет пользовательский интерфейс, обеспечивая высокую производительность и отзывчивость приложений.

- прогрессивность: Vue спроектирован для постепенного внедрения, что позволяет использовать его как для небольших частей проекта, так и для создания полноценных приложений.

- компонентный подход: интерфейс разбивается на независимые компоненты, каждый из которых содержит свою логику, шаблоны и стили.

- шаблоны: фреймворк расширяет стандартный HTML с помощью декларативного синтаксиса шаблонов, что упрощает создание и управление пользовательскими интерфейсами.

К основным преимуществам Vue можно отнести: легкость освоения и низкий порог входа для начинающих разработчиков, простое интегрирование с другими библиотеками и проектами, активная поддержка.

Сам фреймворк может применяться в разработке адаптивного интерфейса и приложений с высокой нагрузкой (таких как интернет-магазины и информационные порталы).

## 2.4. Module Federation

Module Federation – это продвинутая функция в Webpack 5, которая позволяет JavaScript-приложениям динамически загружать код из других приложений во время выполнения. Это облегчает совместное использование кода и ресурсов между различными проектами, упрощая управление и разработку сложных приложений. [12]

Webpack, в свою очередь, представляет собой инструмент с открытым исходным кодом для сборки модулей JavaScript. Он анализирует модули приложения, создает граф зависимостей, затем собирает модули в правильном порядке в один или более бандл (bundle), на который может ссылаться HTML-файл с сайтом. [13]

Данный инструмент также используется для адаптации кода к старым версиям браузера, перевода кода (транспилиции) на TypeScript или «чистый» JavaScript, и более легкого подключения кода JavaScript к HTML-страницы. [14]

У Module Federation есть следующие преимущества:

- улучшенная модульность: приложения могут быть разделены на более мелкие автономные модули, которые можно независимо от других разрабатывать, тестировать и развертывать.

- ускоренная сборка: поддержка инструментов, таких как Rspack, ускоряет сборку приложений, улучшая производительность разработки. Rspack – это высокопроизводительный упаковщик (бандлер) JavaScript, написанный на языке программирования Rust. [15]

### 3. Подходы взаимодействия микрофронтенд приложений

Среди подходов взаимодействия между микрофронтенд приложениями можно выделить:

- взаимодействие через событие
- через общее хранилище
- iframe и пост-сообщения

#### 3.1. Взаимодействие через событие

В этом подходе микрофронты обмениваются данными с помощью событийной системы, например, EventEmitter. Одни микрофронтенд генерирует события, а другие «подписываются» на них и выполняют какие-либо действия.

Преимущества:

- слабая связанность между микрофронтендами
- упрощенная интеграция новых компонентов

Недостатки:

- необходимость управления жизненным циклом событий

#### 3.2. Через общее хранилище

В данном случае, для обмена информации можно использовать временное (session storage) или постоянное (local storage) веб-хранилище. [16]

Также можно использовать библиотеку Redux для управления состоянием данных и пользовательским интерфейсом.

Преимущества:

- согласованное управление состоянием
- предсказуемость поведения приложения

Недостатки:

- усложнение архитектуры приложения
- необходимость синхронизации между командами

### 3.3. Iframe и пост-сообщения

С помощью Iframe можно расположить на сайте, например, форму, которая будет отправлять данные родительскому компоненту. Для этого необходимо использовать пост-сообщения с помощью функции `postMessage`. Родительский элемент будет выступать в роли слушателя события, ожидая отправки сообщения от дочернего. [17]

Данный способ обеспечивает высокую степень изоляции и безопасности. Однако этот подход может усложнить интеграцию и привести к проблемам с производительностью, в частности к увеличению времени загрузки страницы и задержке взаимодействия. Связано это в первую очередь с особенностями Iframe: изоляция компонентов требует дополнительных системных ресурсов, перерисовка и рендеринг происходит после каждого изменения в встраиваемой странице.

Преимущества:

- высокая степень изоляции компонентов
- безопасность взаимодействия

Недостатки:

- сложность интеграции и стилизации
- потенциальные проблемы с производительностью.

## 4. Подходы к сборке микрофронтендов

Сборка микрофронтенда может происходить как на стороне клиента, так и на стороне сервера. От выбранного подхода будет зависеть скорость и работа встраиваемого блока.

### 4.1. Композиция на стороне клиента

В данном подходе микрофронтенды собираются в единое приложение прямо в браузере. Все модули загружаются динамически, обычно через механизмы языка программирования JavaScript.

Для реализации можно использовать Single-SPA (для управления жизненным циклом компонентов) и Module Federation (для динамического подключения модулей в браузере).

Преимущества:

- простота масштабирования
- возможность использовать различные фреймворки
- независимость развертывания (можно обновлять один микрофронт, не затрагивая другие)

Недостатки:

- необходимость загружать несколько фреймворков и библиотек
- сложность управления общими состояниями

#### 4.2. Композиция на стороне сервера

Сборка микрофронтендов происходит на сервере, с использованием всех необходимых модулей и библиотек. Клиент в итоге получает готовую страницу.

Для реализации можно использовать Tailor (инструмент для серверной сборки) и ESI (Edge-side includes – технология, позволяющая вставлять динамические компоненты в страницу на уровне сервера).

Преимущества:

- получение клиентом готовой страницы

Недостатки:

- меньшая гибкость
- более высокая нагрузка на сервер

#### 4.3. Статическая сборка

Все микрофронтенды собираются в единое приложение еще до развертывания. Готовое приложение, собранное с помощью сборщиков (например, Webpack), размещается на сервере и отправляется клиенту.

Преимущества:

- клиент сразу получает оптимизированное приложение

- упрощенная архитектура (всё находится в одном месте)

Недостатки:

- потеря независимости модулей (при изменении одного модуля требуется пересборка всего приложения)
- меньшая гибкость при использовании разных технологий

## **ЗАКЛЮЧЕНИЕ**

В процессе выполнения научно-исследовательской работы были освоены индикаторы ПК-7.1, ПК-7.2, ПК-7.4 компетенции ПК-7, и решены все поставленные задачи:

- был проведен анализ требований к разрабатываемой информационной системе разработки, взаимодействия и сборки микрофронтенд приложений.

- были согласованы и утверждены требования к разрабатываемой информационной системе разработки, взаимодействия и сборки микрофронтенд приложений.

- был проведен анализ имеющихся систем-аналогов;

- был проведен анализ имеющихся технологий проектирования архитектур информационных систем и сделано обоснование выбора используемой технологии;

- был разработан информационно-логический проект информационной системы разработки, взаимодействия и сборки микрофронтенд приложений.;

- было сделано описание информационно-логического проекта информационной системы разработки, взаимодействия и сборки микрофронтенд приложений.;

- был проведен анализ имеющихся технологий проектирования дизайна информационных систем и сделано обоснование выбора используемой технологии;

- был разработан дизайн информационной системы разработки, взаимодействия и сборки микрофронтенд приложений.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Что такое веб-сайт, как он работает и как создают сайты [Электронный ресурс] // URL: <https://skillbox.ru/media/marketing/что-такое-vebsayt-kak-on-rabotaet-i-kak-sozdayut-sayty/> (Дата обращения: 27.12.2024)
2. Фронтенд или бэкенд: по какому пути в разработке пойти? [Электронный ресурс] // URL: <https://practicum.yandex.ru/blog/chem-otlichaetsya-backend-i-frontend/> (Дата обращения: 26.12.2024)
3. Микрофронтенд: что это такое и зачем он нужен? [Электронный ресурс] // URL: <https://proglib.io/p/mikrofrontend-что-это-такое-i-zachem-on-nuzhen-2023-07-13> (Дата обращения: 26.12.2024)
4. Микрофронтенды. Подходы, технологии, способы реализации... [Электронный ресурс] // URL: <https://www.youtube.com/watch?v=WV1ZrrRxYJ4> (Дата обращения: 27.12.2024)
5. Micro Frontends with Podium [Электронный ресурс] // URL: <https://bjerkek.medium.com/micro-frontends-with-podium-e772abda9fe> (Дата обращения: 27.12.2024)
6. Podium.io [Электронный ресурс] // URL: <https://podium-lib.io/> (Дата обращения: 27.12.2024)
7. Angular (фреймворк) [Электронный ресурс] // URL: [https://ru.wikipedia.org/wiki/Angular\\_\(фреймворк\)](https://ru.wikipedia.org/wiki/Angular_(фреймворк)) (Дата обращения: 27.12.2024)
8. Introduction to Angular concepts [Электронный ресурс] // URL: <https://v17.angular.io/guide/architecture> (Дата обращения: 27.12.2024)
9. React [Электронный ресурс] // URL: <https://ru.wikipedia.org/wiki/React> (Дата обращения: 27.12.2024)
10. Introduction Hooks [Электронный ресурс] // URL: <https://legacy.reactjs.org/docs/hooks-intro.html> (Дата обращения: 27.12.2024)

11. Vue.js [Электронный ресурс] // URL: <https://ru.wikipedia.org/wiki/Vue.js>  
(Дата обращения: 27.12.2024)
12. Module Federation [Электронный ресурс] // URL: <https://webpack.js.org/concepts/module-federation/> (Дата обращения: 27.12.2024)
13. Webpack: руководство для начинающих [Электронный ресурс] // URL: <https://habr.com/ru/articles/514838/> (Дата обращения: 27.12.2024)
14. Webpack – что это за инструмент, настройка для новичков [Электронный ресурс] // URL: <https://blog.skillfactory.ru/glossary/webpack/> (Дата обращения: 27.12.2024)
15. Rspack [Электронный ресурс] // URL: <https://rspack.dev/> (Дата обращения: 27.12.2024).
16. Микрофронтенды [Электронный ресурс] // URL: <https://dev.to/balaevarif/-165c> (Дата обращения: 27.12.2024)
17. Communicating with Iframes [Электронный ресурс] // URL: <https://medium.com/@ariel.salem1989/communicating-with-iframe-712fdc2b4d14> (Дата обращения: 27.12.2024)